EXAM PREP · VIDEO TUTORIAL

Claude Certified Architect
Foundations

A plain-language walkthrough of all five domains, every task statement, the six scenarios, and the judgment the exam rewards — for TPMs, program managers, and AI-curious builders.

Prepared by Jacinth P · Technical Program Manager

Three ideas that run through every domain

The exam is judgment, not trivia. Almost every question is: given this production problem, which fix targets the real root cause? Internalize these three and most answers fall out on their own.

1

Guarantees beat instructions

When a rule MUST hold — verify identity before a refund — enforce it in code: hooks, gates, schemas. Prompts are probabilistic; they fail a small % of the time.

2

Fix the root cause, proportionately

Vague tool descriptions? Improve the descriptions — don't build a routing classifier. The exam loves over-engineered distractors.

3

Context is scarce and leaky

The window fills up, the middle gets lost, summaries drop the numbers. Extract key facts, trim noise, pass information explicitly between agents.

The exam at a glance

720

minimum passing score

on a scaled 100–1,000 range

4 of 6

scenarios per exam

drawn at random from the pool

~6 mo+

hands-on Claude experience

the target candidate profile

Format & scoring

- Multiple choice — 1 correct answer, 3 distractors built to tempt incomplete knowledge
- No guessing penalty — answer every question; blanks count as wrong
- Pass / Fail, judged against a standard set by subject-matter experts

Four core technologies

Claude Code
the agentic coding tool

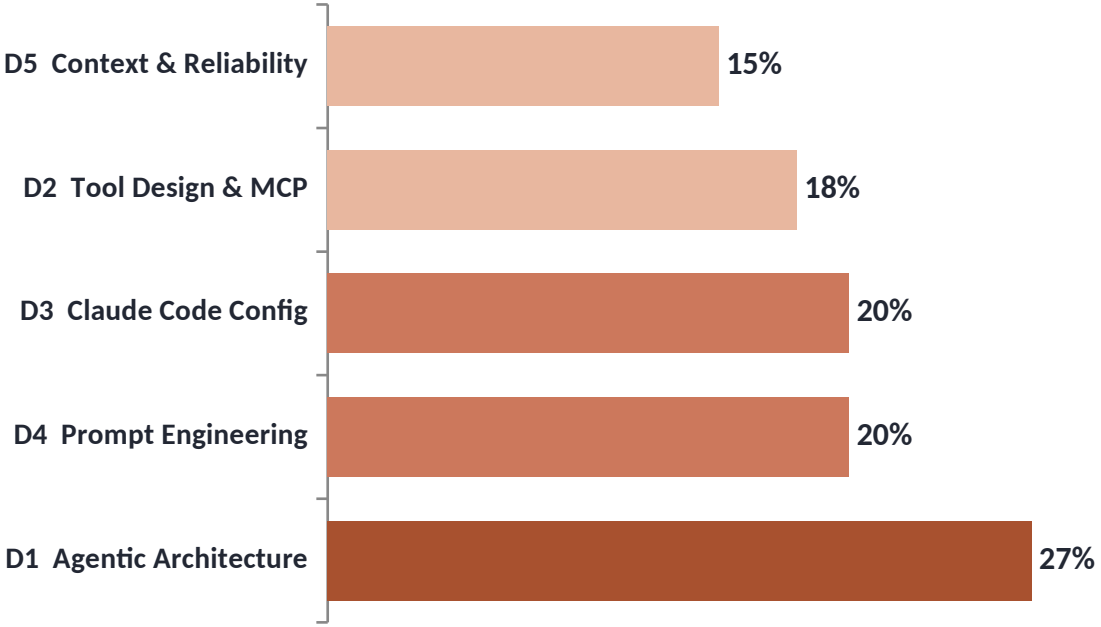
Claude Agent SDK
build your own agents

Claude API
direct programmatic access

MCP
open standard for connections

Where the points sit

Five content domains are tested. Weighting tells you where to invest study time.



Master these first

Domains 1 + 2 = 45%
of the exam, and share one philosophy: agents are only as reliable as the structure around them — tool descriptions, programmatic gates, and explicit context passing.

Every question lives in one of six scenarios

Each is a realistic production system. Knowing each one's shape — what's built, what breaks — lets you anticipate the questions.

1 Customer Support Agent

Resolves returns, billing & accounts end-to-end via four MCP tools.

D1 · D2 · D5

2 Code Generation

A team adopts Claude Code — commands, CLAUDE.md, plan mode.

D3 · D5

3 Multi-Agent Research

A coordinator delegates to search, analysis & synthesis specialists.

D1 · D2 · D5

4 Developer Productivity

Explore unfamiliar, legacy codebases efficiently with built-in tools.

D2 · D3 · D1

5 Claude Code for CI/CD

Automated review with few false positives, no human at the keyboard.

D3 · D4

6 Structured Data Extraction

Unstructured docs to clean, schema-validated JSON.

D4 · D5

Scenario-to-domain map

Each scenario leans on a particular cluster of domains. Spot the scenario and you can predict which skills the question is really testing.

Scenario	Primary domains tested
1 · Customer Support Agent	D1 Orchestration · D2 Tools/MCP · D5 Context & Reliability
2 · Code Generation (Claude Code)	D3 Claude Code Config · D5 Context & Reliability
3 · Multi-Agent Research System	D1 Orchestration · D2 Tools/MCP · D5 Context & Reliability
4 · Developer Productivity	D2 Tools/MCP · D3 Claude Code Config · D1 Orchestration
5 · Claude Code for CI/CD	D3 Claude Code Config · D4 Prompting & Structured Output
6 · Structured Data Extraction	D4 Prompting & Structured Output · D5 Context & Reliability

The distinction the exam loves

Roughly a third of the exam reduces to one judgment: is this rule too important to leave to a probabilistic mechanism?

DETERMINISTIC

Same input → same outcome, every single time.

Code, calculators, locked doors

"block refunds over $500"

Use for: money, security, compliance — anything where one failure is unacceptable

PROBABILISTIC

Usually right, occasionally not.

LLMs — like a skilled employee following written guidance

"you must verify identity first"

Use for: judgment, language, flexibility — where occasional deviation is tolerable

The exam's reflex: when a rule must never break, replace the probabilistic mechanism with a deterministic one.

The vocabulary you'll be tested on

Agent / agentic
An AI that works in a loop: think, act (call a tool), observe, decide, repeat until done.

Tool
A capability the agent invokes — look up an order, search the web, edit a file. Has a name, description & I/O.

MCP
Model Context Protocol — “USB-C for AI.” Servers expose tools (actions) and resources (readable catalogs).

stop_reason
The loop's control signal: “tool_use” = keep going; “end_turn” = finished.

Coordinator / subagent
A lead agent that delegates to specialists with isolated memory.

Hook
Code that intercepts actions before/after a tool runs — deterministic, guaranteed enforcement.

Context window
The model's finite working memory; verbose output degrades quality as it fills.

CLAUDE.md
A plain-text instruction file Claude Code loads automatically — the project's operating manual.

JSON schema
The contract for structured output: required fields, types, and allowed values.

Few-shot
2–4 worked input→output examples in the prompt so the model generalizes the pattern.



DOMAIN 1

Agentic Architecture & Orchestration

27% of the exam · 7 task statements

- 1.1 Design & implement agentic loops
- 1.2 Orchestrate multi-agent systems
- 1.3 Subagent invocation & context passing
- 1.4 Multi-step workflow enforcement
- 1.5 Agent SDK hooks
- 1.6 Task decomposition strategies
- 1.7 Session state, resumption & forking

1

DOMAIN 1 · TASK 1.1

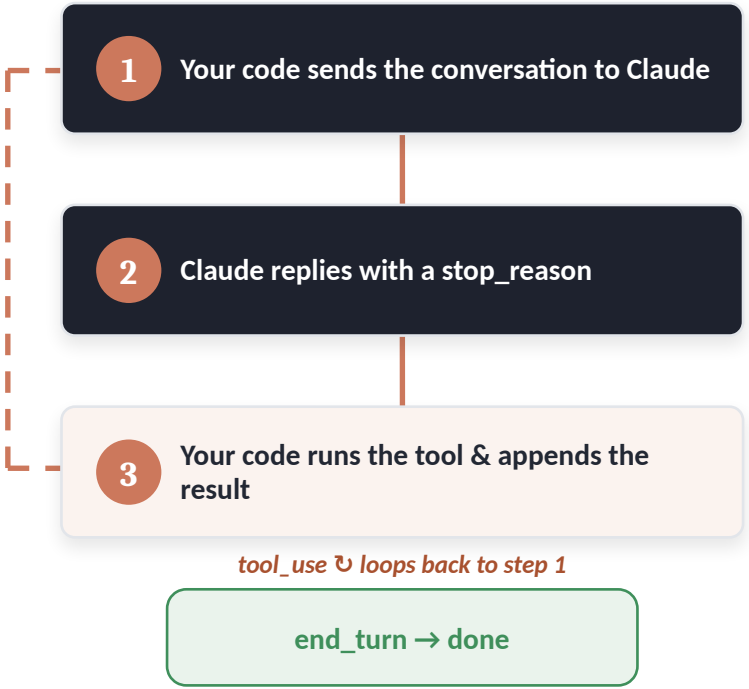
Design & implement agentic loops

27%

An agent is a conversation between your code and Claude that repeats until the job is done. After each reply, your code checks one field — stop_reason — to decide what happens next.

“tool_use”
Run the tool, append the result to history, send it back — the loop continues.

“end_turn”
Claude is finished — exit the loop and present the answer.

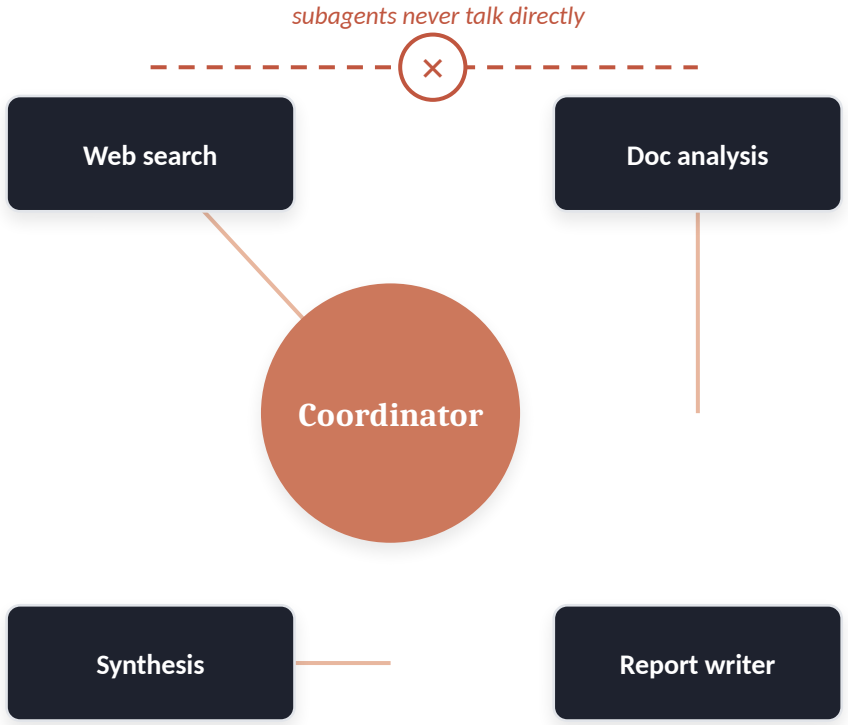


Exam tip: terminate on stop_reason — never by parsing text or hitting an iteration cap. Claude decides which tool; your code executes it.

DOMAIN 1 · TASK 1.2

Orchestrate multi-agent systems

27%



Hub-and-spoke: all communication routes through the coordinator, whose four jobs are —

decompose · delegate · aggregate · evaluate & re-delegate

Isolated context

Subagents don't inherit history — write everything they need into the prompt. The coordinator needs “Task” in allowedTools to delegate at all.

Narrow decomposition = top failure

If whole subtopics are never assigned, each subagent can still “succeed” while the report stays incomplete. Partition scope to avoid duplication.

Goals over procedures

State research goals and quality criteria, not rigid step-by-step scripts — so subagents can adapt.

DOMAIN 1 · TASK 1.3

Subagent invocation & context passing

27%

The mechanics of actually launching subagents in the Claude Agent SDK — there is no shared memory, so everything is explicit.

The Task tool spawns subagents

The coordinator's allowedTools list must include “Task” — or it simply cannot delegate. A classic exam gotcha.

Context must be passed explicitly

Need the web-search findings in the synthesis agent? Paste them into its prompt. Nothing is inherited.

AgentDefinition

Each subagent's job description: purpose, system prompt, and tool restrictions.

Parallel spawning

To run subagents at once, emit multiple Task calls in a single response — not one per turn. This cuts latency.

Structured handoffs preserve attribution

Pass findings as structured data, separating content from metadata (URLs, doc names, pages) so citations survive.

fork_session

Branch independent explorations from a shared baseline — e.g., two refactoring strategies without redoing analysis.

DOMAIN 1 · TASK 1.4

Workflow enforcement & handoffs

27%

The single most important idea in this domain: prompt instructions are probabilistic; programmatic enforcement is deterministic.

Prompt-based guidance

"Always verify identity before issuing refunds"

Followed most of the time — fails a small % of cases.
Unacceptable for money, security, or compliance.

Programmatic gate

A hook blocks process_refund until get_customer has verified the ID

Enforced 100% of the time, no matter what the prompt says.
For rules that must never break.

Also in this task

Multi-concern requests: decompose three issues in one message, investigate each (in parallel, shared context), then synthesize one unified resolution.
Structured handoff summaries: when escalating to a human who can't see the chat, compile customer ID, root cause, refund amount & recommended action — never a cold start.

DOMAIN 1 · TASK 1.5

Agent SDK hooks: intercept & normalize

27%

Hooks are checkpoints around tool calls — deterministic, guaranteed enforcement. Two patterns matter.

PostToolUse (after)

Transform tool results before Claude sees them.

Three backends return dates as Unix timestamps, ISO strings, and status codes — a PostToolUse hook normalizes them all to one format so the agent never juggles inconsistencies.

Interception (before)

Inspect outgoing tool calls and block policy violations.

Any process_refund above $500 is blocked and redirected to human escalation — guaranteed, every time, regardless of what the prompt says.

Exam tip: "guaranteed compliance required" → hook. Distractors will offer prompt wording, few-shot examples, or confidence thresholds instead.

DOMAIN 1 · TASKS 1.6 & 1.7

Task decomposition & session state

27%

Match the decomposition style to the work — and manage sessions so stale context never poisons a resumed run.

Prompt chaining

A fixed sequential pipeline of focused passes. Best for predictable, multi-aspect work where the steps are known.

e.g. review each file individually, then one cross-file integration pass.

Dynamic adaptive decomposition

Generate the next subtasks from what each step discovers. Best for open-ended investigation you can't script upfront.

e.g. add tests to a legacy codebase: map structure, find high-impact areas, adapt as dependencies surface.

Session situation	Right move
Prior context still mostly valid; a few files changed	Resume the session; list the changed files for targeted re-analysis
Prior tool results largely outdated	Start a new session seeded with a structured summary of validated findings
Compare two divergent approaches from one analysis	**fork_session into parallel branches**



DOMAIN 2

Tool Design & MCP Integration

18% of the exam · 5 task statements

- 2.1 Design effective tool interfaces
- 2.2 Structured error responses
- 2.3 Distribute tools & configure tool_choice
- 2.4 Integrate MCP servers
- 2.5 Built-in tools (Grep, Glob, Read, Write, Edit, Bash)

2

DOMAIN 2 · TASK 2.1

Design effective tool interfaces

18%

Tool descriptions are the primary mechanism for tool selection. The model reads exactly this text when deciding which tool to call — richer text, better decisions.

× WEAK — causes misrouting

"Retrieves customer information."

✓ STRONG

Looks up a customer by email or ID (CUST-12345). Returns name, status, tier. For a specific purchase, use lookup_order instead — this tool has no order data.

Remedies the exam tests, in order of effort

Expand descriptions first — purpose, inputs, examples, edge cases, boundaries. The correct "first step."
Rename to remove overlap — analyze_content → extract_web_results.
Split generic tools — one analyze_document → extract_data_points, summarize_content, verify_claim.
Audit the system prompt — "always analyze documents" can bias the model toward any tool with "document" in its name.

DOMAIN 2 · TASK 2.2

Structured error responses

18%

A bare “Operation failed” is useless. Return errorCategory + isRetryable so the agent knows whether to retry, rephrase, apologize, or escalate.

Category	Example	Retryable?	Right agent behavior
Transient	Timeout; service temporarily unavailable	Yes	Retry (subagents try local recovery first)
Validation	Malformed order ID; missing input	No (as-is)	Fix the input and try again
Business	Refund exceeds policy limit	No	Explain the policy; offer alternatives or escalate
Permission	Agent lacks access rights	No	Escalate or route to an authorized path

Crucial distinction: “the search failed” (needs a retry decision) is NOT “the search succeeded and found nothing” (a valid empty result). Conflating them causes wasted retries — or silent data gaps.

DOMAIN 2 · TASK 2.3

Distribute tools & tool_choice

18%

Fewer tools = better decisions. ~4–5 well-described tools beat 18; agents with tools outside their specialty tend to misuse them.

Scoped access pattern

Each subagent gets only its role's tools, plus narrowly scoped cross-role tools for high-frequency needs.

Flagship example: synthesis agent needs frequent fact checks (85% simple). Give it a scoped verify_fact tool; route the complex 15% through the coordinator.

tool_choice	Behavior / when to use
“auto”	May call a tool or just reply — normal conversation
“any”	MUST call some tool — structured output when the right schema depends on the input
{type:“tool”}	MUST call that specific tool — force a step to run first

Constrain rather than trust: replace a generic fetch_url with a load_document tool that validates URLs — the tool itself enforces the boundary.
Wrong answers the exam offers: give it all search tools (over-provisioning), batch the verifications (blocking dependencies), or speculative caching (can't predict needs).

DOMAIN 2 · TASKS 2.4 & 2.5

MCP integration & built-in tools

18%

Two things to get right: which built-in tool to reach for, and where MCP servers live.

Built-in tool	What it does / when to pick it (Grep = contents · Glob = names)
Grep	Searches file CONTENTS for patterns — find all callers of a function
Glob	Matches file NAMES / paths — **/*.test.tsx for every test file
Read / Write / Edit	Load a file · write a whole file · targeted edit via unique anchor (else Read+Write)
Bash	Builds, tests, git operations, scripting

MCP server configuration

.mcp.json = project, shared with the team via version control **~/.claude.json** = personal, not shared
Secrets via ${ENV_VAR} so config can be committed · resources = readable catalogs, tools = actions · buy before build (community servers for standard integrations like Jira).



DOMAIN 3

Claude Code Configuration & Workflows

20% of the exam · 6 task statements

- 3.1 CLAUDE.md hierarchy & scoping
- 3.2 Custom slash commands & skills
- 3.3 Path-specific rules
- 3.4 Plan mode vs direct execution
- 3.5 Iterative refinement techniques
- 3.6 Claude Code in CI/CD pipelines

3

DOMAIN 3 · TASK 3.1

CLAUDE.md hierarchy & scoping

20%

Standing instructions Claude Code loads automatically — the project's operating manual. The level decides who they reach.

User

`~/.claude/CLAUDE.md`

Everything you do — personal only

not shared

Project

`repo CLAUDE.md`

Everyone in the repo — via version control

shared

Directory

`subfolder CLAUDE.md`

Work within that subdirectory

scoped

```
# Project conventions

- amounts stored in cents
- every function needs a test
- British English in UI text
- never touch /legacy without asking
```

@import keeps it modular · .claude/rules/ splits topics · /memory shows what's loaded

Classic gotcha: a teammate clones the repo and Claude ignores the standards — they were written at user level (~/.claude). Move them to project level.

DOMAIN 3 · TASKS 3.2 & 3.3

Slash commands, skills & path rules

20%

Commands & skills

.claude/commands/ = saved prompts shared with everyone who clones (the answer for team-wide /review).
~/.claude/commands/ = personal.
Skills live in .claude/skills/ as SKILL.md playbooks; frontmatter sets context: fork, allowed-tools, argument-hint.

Path-scoped rule

```
---
paths: ["**/*.test.tsx"]
---
Test names describe behavior;
mock external services.
```

Decision rules

CLAUDE.md vs skills: CLAUDE.md is for always-loaded universal standards; skills are on-demand, task-specific workflows.
Path rules vs directory CLAUDE.md: test files sit scattered next to the code they test. A directory file can't cover them all — one rule with paths: ["**/*.test.tsx"] applies everywhere, automatically and deterministically.
Analogy: path rules are motion-sensor lights — they switch on exactly where you're working. A monolithic CLAUDE.md leaves every light in the building on.

DOMAIN 3 · TASKS 3.4 & 3.5

Plan mode & iterative refinement

20%

Plan mode

Explore & design BEFORE changing anything. For large-scale changes, multiple valid approaches, architectural decisions, multi-file edits.

e.g. monolith → microservices; a migration touching 45+ files.

Direct execution

Simple, well-scoped changes you already understand. Just make the change.

e.g. a single-file bug fix with a clear stack trace; adding one date-validation conditional.

Iterative refinement techniques

Concrete examples (2–3) Show exact before → after when prose is read inconsistently.

Test-driven iteration Write tests first; share failures as precise, objective feedback.

The interview pattern Have Claude interview YOU before building in an unfamiliar domain.

Batch interacting fixes Describe interacting issues together; sequence independent ones.

DOMAIN 3 · TASK 3.6

Claude Code in CI/CD pipelines

20%

Running with no human at the keyboard requires specific flags and context discipline.

Mechanism	Purpose
-p (--print)	Non-interactive: process, print, exit. Without it the CI job hangs forever waiting for input.
--output-format json	+ --json-schema forces machine-parseable findings the pipeline posts as inline PR comments.
CLAUDE.md in CI	Supplies project context — testing standards, review criteria — improving generation quality.
Independent instance	The session that generated code is biased reviewing it; a fresh instance catches more.
Prior findings on re-review	Include earlier findings; report only new or still-unaddressed issues to avoid duplicates.

Invented-flag trap: CLAUDE_HEADLESS and --batch do not exist. The correct answer is claude -p "...".

DOMAIN 4

Prompt Engineering & Structured Output

20% of the exam · 6 task statements

- 4.1 Explicit criteria to reduce false positives
- 4.2 Few-shot prompting
- 4.3 Structured output & JSON schemas
- 4.4 Validation, retry & feedback loops
- 4.5 Batch processing strategies
- 4.6 Multi-instance & multi-pass review

DOMAIN 4 · TASKS 4.1 & 4.2

Explicit criteria & few-shot prompting

20%

Explicit criteria cut false positives

“Be conservative” doesn't work

Vague modifiers fail. Specific categorical criteria succeed: “flag only when the claimed behavior contradicts the actual code.”

Anchor severity with concrete code examples; temporarily disable a noisy category while you fix its prompt.

Few-shot, in an extraction prompt

```
In: “Pd $1,200.50 to Acme on 3/4/25”
Out: {vendor:“Acme”, amount:1200.50,
      date:“2025-03-04”}
In: “Invoice from Beta, amount TBC”
Out: {vendor:“Beta”, amount:null}
```

Few-shot is the most effective fix when detailed instructions still produce inconsistent results

2–4 worked examples teach judgment that generalizes — not literal matching. Example 2 above teaches the most important behavior of all: when information is genuinely missing, write null — don't invent a number. Where it shines: **ambiguous-case handling, format consistency, false-positive reduction, and hallucination reduction across varied document structures.**

DOMAIN 4 · TASK 4.3

Structured output & JSON schemas

20%

The reliable trick: define a “tool” whose input parameters ARE your desired output schema, and read the data from the model's tool_use call. This eliminates JSON ***syntax*** errors entirely.

The limit you must know: schemas kill syntax errors but NOT semantic errors — line items that don't sum, or values in the wrong field. Those still need validation.

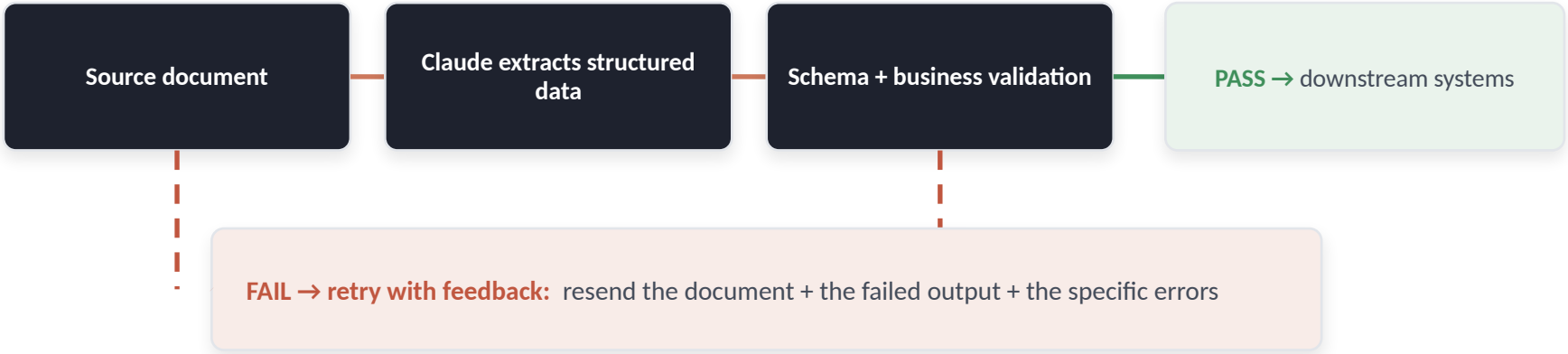
Schema design rules that prevent hallucination

- **Make fields nullable / optional —** If a field is required, the model fabricates a value to satisfy the schema — the single most important schema fact on the exam.
- **Extensible enums —** Add an “unclear” value for ambiguous cases and an “other” value paired with a free-text detail field.
- **Pair normalization with strict schemas —** Tell the model how to standardize inconsistent dates & currencies alongside the schema.

DOMAIN 4 · TASK 4.4

Validation, retry & feedback loops

20%



- **Specific feedback steers self-correction —** Blind retries don't. Send the exact validation errors back.
- **Know when retry is futile —** Retries fix format & structure — NEVER information that's absent from the source (e.g. it lives in a document you didn't provide).
- **Self-correcting schema design —** Extract calculated_total alongside stated_total so discrepancies flag themselves; add a conflict_detected boolean.

DOMAIN 4 · TASKS 4.5 & 4.6

Batch processing & multi-pass review

20%

Message Batches API

50% cheaper · ≤24h window · no latency SLA · no multi-turn tool calling.

Right for overnight reports & nightly test generation. Wrong for a pre-merge check a developer is waiting on.

custom_id correlates requests and lets you resubmit only the failed documents.

Multi-instance & multi-pass review

Self-review is structurally weak

A fresh, independent instance catches subtler issues than self-review or extended thinking.

Large reviews: per-file passes + a separate cross-file integration pass — bigger context windows do NOT fix attention quality.

Workload	Right API	Why
Pre-merge check (developers waiting)	Synchronous	Blocking — “usually fast” isn't good enough without an SLA
Overnight technical-debt report	Batch	Latency-tolerant; pocket the 50% savings
Weekly audit / nightly test generation	Batch	Nobody is waiting on the result

DOMAIN 5

Context Management & Reliability

15% of the exam · 6 task statements

- 5.1 Preserve critical information
- 5.2 Escalation & ambiguity resolution
- 5.3 Error propagation across agents
- 5.4 Large-codebase context management
- 5.5 Human review & confidence calibration
- 5.6 Information provenance & uncertainty

DOMAIN 5 · TASK 5.1

Preserve critical information

15%

Progressive summarization erodes precision — each round strips detail. Numbers, dates & customer expectations are the first casualties.

Original: “Dana Lee (order #88231) was promised a $847.50 refund by March 3 after a second damaged delivery.”

Summary 1: “Dana Lee was promised a refund for delivery issues.”

Summary 2: “A customer had delivery problems; a refund was discussed.”

- **The “case facts” block —** Extract transactional facts (amounts, dates, order #s, statuses) into a persistent structured block in every prompt — outside the summarized history.
- **Lost in the middle —** Models process the beginning & end reliably but skip the middle. Put a key-findings summary at the TOP, with section headers.
- **Trim tool-output bloat —** An order lookup returns 40+ fields when 5 matter. Trim to relevant fields before they accumulate. (And: the API is stateless — pass full history each call.)

DOMAIN 5 · TASK 5.2

Escalation & ambiguity resolution

15%

Escalate for the right reasons — and never on signals that don't correlate with case complexity.

Signal	Reliable trigger?	Why
Customer explicitly demands a human	**Yes — honor it**	Don't “investigate first”; that disrespects an explicit preference
Policy gap / exception required	**Yes**	The agent must not invent policy
Customer frustrated but issue is simple	**No — resolve it**	Acknowledge, offer the fix; escalate only if they reiterate
Negative sentiment score	**No**	Sentiment doesn't correlate with complexity
Model self-reported confidence	**No**	LLM confidence is poorly calibrated; worst on hard cases

The calibration fix: an agent escalating easy cases while fumbling hard ones (55% vs 80% target) needs explicit escalation criteria + few-shot examples — not confidence thresholds, a classifier, or sentiment. Multiple matches? Ask for more identifiers — never pick “most likely.”

DOMAIN 5 · TASK 5.3

Error propagation across agents

15%

When a subagent fails (say, a web-search timeout), what flows back to the coordinator decides whether intelligent recovery is possible.

Approach	Verdict	Why
Structured error context	CORRECT	Failure type, attempted query, partial results, alternatives — the coordinator can retry, swap source, or annotate the gap
Retry, then return generic “unavailable”	Anti-pattern	The generic status hides everything the coordinator needs
Return empty results as success	Anti-pattern	Silent suppression — the report ships with invisible holes
Propagate exception; kill the workflow	Anti-pattern	One recoverable failure kills the entire research run

Coverage annotations: synthesis output should mark which findings are well-supported and which topic areas have gaps. Honest uncertainty beats false completeness.

DOMAIN 5 · TASKS 5.4 & 5.5

Large codebases & human review

15%

Codebase exploration (5.4)

Recognize context degradation

When the model starts answering from “typical patterns” instead of the classes it discovered — working memory is squeezed.

Scratchpad files

Write key findings to a file and consult it later — external memory that survives context pressure.

Delegate verbose work

Spawn subagents for noisy investigations so only summaries return. Use /compact when discovery fills the window.

Human review & calibration (5.5)

Aggregate metrics hide failure

“97% overall” can mask 60% on one document type. Validate by document type AND field segment.

Stratified random sampling

Keep auditing HIGH-confidence extractions too — it measures the true error rate where you've stopped looking.

Calibrated confidence routing

Output field-level scores, calibrate on a labeled set, route low-confidence / contradictory cases to humans.

DOMAIN 5 · TASK 5.6

Provenance & uncertainty in synthesis

15%

Attribution dies in summarization unless you force structure. Preserve who said what — all the way through synthesis.

Force claim-source mappings

Subagents must output claim + source URL/doc name + relevant excerpt; downstream agents preserve and merge them.

Conflicting statistics

Annotate the conflict with both values and their sources — never arbitrarily pick one. Let the coordinator reconcile.

Temporal traps

A 2021 figure and a 2024 figure are a trend, not a contradiction. Require dates in structured output.

Separate established from contested

Explicitly split well-established findings from contested ones, preserving each source's characterization.

Render content natively

Financial data as tables, news as prose, technical findings as lists — don't flatten everything into one format.

How to think through any question

The rhythm: root cause + the simplest reliable mechanism wins.

Three distractor archetypes — learn to eliminate them on sight:

More prompting where enforcement is needed

Prompt wording & few-shot are probabilistic. If money, security, or ordering must be guaranteed, the answer is a hook or gate.

More infrastructure where prompting suffices

Classifiers, routing layers, second models — over-engineered. A clearer description or explicit criteria usually fixes the root cause.

Features that don't exist

Invented flags & config: CLAUDE_HEADLESS, --batch, a config.json commands array. Spot the fiction and eliminate it instantly.

Cheat sheet: facts to memorize

stop_reason
tool_use = run & continue; end_turn = done. Never parse text or rely on iteration caps.

Enforcement
Guaranteed rules → hooks/gates; preferences → prompts.

tool_choice
auto = optional; any = must call some tool; {type:tool} = that tool.

CLAUDE.md
User (personal) · Project (shared) · Directory (scoped); path rules via paths: glob; /memory to debug.

Batches API
50% cheaper, ≤24h, NO latency SLA; custom_id correlates & resubmits failures.

Escalation
Explicit human request, policy gap, no progress. NOT sentiment, NOT self-confidence.

Subagents
Isolated memory — pass context explicitly; coordinator needs "Task" in allowedTools; parallel = multiple Task calls in one response.

Tools
Driven by descriptions; ~4–5 beat 18; first fix = better descriptions. Errors: transient/validation/business/permission.

MCP & built-ins
.mcp.json shared / ~/.claude.json personal; secrets via ${ENV_VAR}. Grep = contents, Glob = names.

Schemas
Tool use kills syntax not semantic errors; nullable fields prevent fabrication; enums need unclear/other.

Review & context
Fresh instance > self-review; large PRs → per-file + integration pass; "case facts" block survives summarization.

Provenance
Claim-source mappings through synthesis; annotate conflicts; include dates to avoid temporal "contradictions."

A two-week study plan

Days	Focus	Activity
1–2	**Foundations**	Read the front matter & glossary; spend extra time on the basics if terms feel shaky; make flashcards.
3–5	**Domains 1 & 2 (45%)**	In Claude Code, watch the loop in action — the think → act → observe rhythm.
6–7	**Domain 3 hands-on**	Create a real project CLAUDE.md, one slash command, one path-scoped rule; try plan vs direct.
8–9	**Domains 4 & 5**	Practice few-shot & explicit criteria; design a schema with nullable fields and an “other” enum.
10–11	**Sample questions**	For each, articulate WHY each distractor fails before reading the explanation.
12–13	**Practice exam**	Take it; re-study every domain where you missed questions.
14	**Final review**	Re-read the cheat sheet and the three recurring themes; rest.

End

Thank you for watching

Keep in Touch

Subscribe to Youtube

Connect on Linkedin

Good luck.

Prepared by Jacinth P · Technical Program Manager